

AGENTIC AI ENGINEERING PROGRAM

Capstone Project Handbook

Four Real-World LangChain & Streamlit Builds



Prepared for Program Participants

August 2026

About This Handbook

This handbook presents four capstone-project briefs for the Agentic AI Engineering track. Each brief is written as a business case: a fictional company with a real operational pain point, an assistant that LangChain and Streamlit are used to build, and a functional specification broken into numbered modules.

Every project follows the same shape so that comparisons are easy — a case study, project objective, system architecture, functional requirements by module, Streamlit interface requirements, recommended LangChain components, example workflows, and expected deliverables. Read a brief end-to-end before starting implementation; the modules are numbered in a sensible build order, but the architecture diagram at the top of each project is the fastest way to see how the pieces fit together.

How to Use This Document

- Pick one project as your capstone — each is scoped for a single team over the course duration.
- Treat “Optional” modules as stretch goals once the core pipeline works end-to-end.
- The “Suggested / Recommended Components” lists are a checklist for your final submission, not a mandate to use every listed tool in every module.
- Example prompts in each module are illustrative — your demo should cover at least one of them per module.

At a Glance

▶ Project 1 — Enterprise Research & Report Generation AI Assistant	▶ Project 2 — Enterprise Operations AI Assistant (Multi-Agent)
▶ Project 3 — AI Investment & Financial Research Assistant	▶ Project 4 — AI Personal Knowledge & Productivity Assistant

Table of Contents

- 1. Enterprise Research & Report Generation AI Assistant**
- 2. Enterprise Operations AI Assistant** — *Multi-Agent Business Automation Platform*
- 3. AI Investment & Financial Research Assistant**
- 4. AI Personal Knowledge & Productivity Assistant** — *"Second Brain"*

Each project is self-contained and includes a case study, objective, system architecture, module-by-module functional requirements, Streamlit interface specification, recommended components, example workflows, and expected deliverables.

CAPSTONE PROJECT 1

Enterprise Research & Report Generation AI Assistant

Case Study

ABC Consulting Pvt. Ltd. is a business consulting firm that spends significant time researching industries, competitors, market trends, and company reports for clients.

Currently, consultants manually search websites, read PDFs, compare multiple sources, generate summaries, write reports, and email reports to clients — a process that takes several hours for every client.

The company wants an AI-powered Enterprise Research Assistant that can automatically perform research, analyze documents, remember previous client discussions, and generate professional reports. The assistant should function like an intelligent research analyst.

Project Objective

Develop an AI Enterprise Research Assistant capable of:

- Understanding user queries
- Searching the web
- Searching uploaded company documents
- Combining information from multiple sources
- Remembering previous conversations
- Generating structured reports
- Exporting reports
- Sending reports through Gmail
- Running completely through a Streamlit application

Functional Requirements

Module 1 — AI Chat Assistant

The application shall provide a conversational interface. The assistant should understand the request naturally.

“Research Tesla AI Strategy”

Module 2 — Internet Research

The assistant should use an internet search tool and gather current information before answering.

“Latest NVIDIA News”

“Current Google Stock”

"India's AI Market"

"OpenAI Competitors"

Module 3 — Wikipedia Research

The assistant should retrieve background knowledge from Wikipedia.

"Explain Artificial General Intelligence"

"History of Microsoft"

Module 4 — Document Research (RAG)

Users should be able to upload PDFs and TXT documents (annual reports, research papers, policy documents, company manuals). The assistant should answer questions only from uploaded documents whenever appropriate.

Module 5 — Multi-Source Research

The assistant should combine the internet, Wikipedia, and uploaded PDFs to prepare a single answer.

"Research Tesla's AI strategy using latest news and the uploaded annual report."

Module 6 — Structured Output

Every report should follow a fixed structure — Title, Executive Summary, Key Findings, Strengths, Weaknesses, Future Opportunities, Conclusion, and References — implemented using Pydantic structured output.

Module 7 — Parallel Agent

Research multiple topics simultaneously, with each company researched independently before results are combined.

"Research Google, Microsoft, Amazon, and OpenAI."

Module 8 — Sequential Chain

A pipeline that moves through: Research → Summarize → Generate Report → Generate Executive Summary → Prepare Email Draft.

Module 9 — Memory

Short-term memory retains the ongoing conversation. Persistent memory remembers previous client conversations. Long-term memory retains client name, preferred report style, and frequently researched industries.

Module 10 — Python Tool

Students should create a Python tool capable of calculating statistics, creating comparison tables, and performing simple data analysis.

"Compare Revenue Growth"

Module 11 — Gmail Integration

The assistant should generate or send the final report to manager@company.com.

Module 12 — Google Drive Integration (Optional)

Allow users to upload reports, search reports, and retrieve previous reports.

Streamlit Application Requirements

Sidebar

- Upload PDF
- Upload TXT
- Create Knowledge Base
- Clear Conversation
- Previous Sessions

Main Screen

- Chat interface
- Research results
- Generated reports

Expandable Sections

- Internet Sources
- PDF Sources
- Final Summary

Download Options

- TXT
- PDF

Recommended LangChain Components

▶ Chat Models	▶ Prompt Templates
▶ ChatPromptTemplate	▶ Structured Output
▶ RunnableLambda	▶ RunnableParallel
▶ RunnableBranch	▶ Custom Tools
▶ create_agent()	▶ Memory
▶ ChromaDB Retriever	

Expected Workflow



Optional Extensions

Students may additionally implement:

- Citation support
- Multi-PDF research
- Conversation export
- Dark mode
- Report history
- AI-generated executive presentation
- Voice input
- Research timeline visualization

Expected Deliverables

- | | |
|---------------------|----------------------------|
| ▶ Source Code | ▶ Streamlit Application |
| ▶ GitHub Repository | ▶ Project Documentation |
| ▶ Requirements File | ▶ Sample Company Documents |

► README

CAPSTONE PROJECT 2

Enterprise Operations AI Assistant

Multi-Agent Business Automation Platform

Case Study

NovaTech Solutions Pvt. Ltd. is a growing software company with more than 500 employees. Every day, employees interact with multiple departments such as HR, Operations, Finance, IT Support, and Administration.

Currently, employees spend a significant amount of time searching company policies, sending repetitive emails, finding documents, preparing meeting summaries, looking for project information, asking HR questions, managing tasks, and retrieving company forms.

The company wants to build an Enterprise Operations AI Assistant capable of acting as a digital employee assistant. Instead of one chatbot, the organization wants multiple specialized AI agents working together to complete employee requests.

The assistant should understand user requests, decide which department should handle them, use appropriate tools, and provide accurate responses. The complete application must be developed using LangChain and deployed using Streamlit.

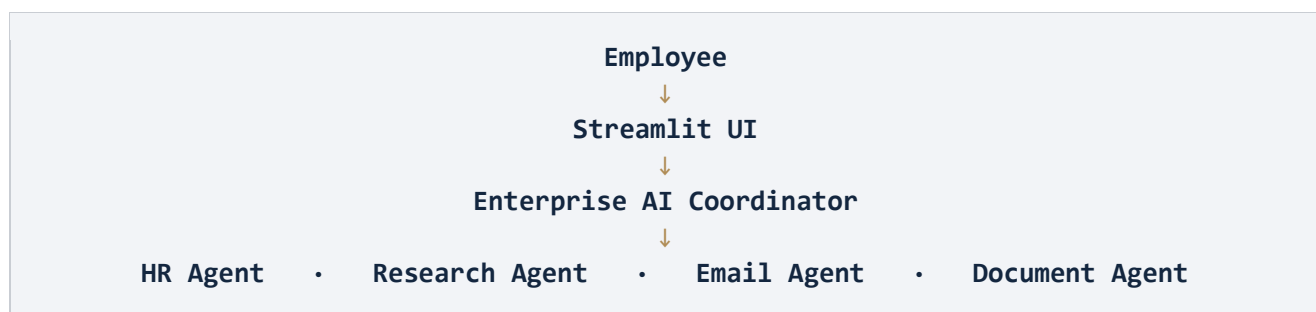
Project Objective

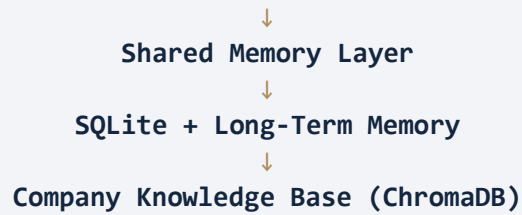
Develop an Enterprise AI Assistant capable of automating common business operations using multiple AI agents, tools, memory, and document retrieval.

Business Scenario

An employee may ask: "I need the leave policy, summarize yesterday's HR email, create a leave request email, and save it in Google Drive." Instead of one prompt doing everything, the application should intelligently coordinate multiple agents to complete the task.

System Architecture





Functional Requirements

Module 1 — Employee Chat Interface

The system shall provide a conversational chat interface where employees can interact naturally.

"What is our work from home policy?"

"Prepare a leave request email."

"Summarize yesterday's HR updates."

Module 2 — HR Agent

Develop an HR Agent responsible for handling employee-related questions, retrieving information from uploaded HR policy documents using RAG.

- Leave Policy
- Attendance Policy
- Work From Home Rules
- Holiday List
- Employee Benefits
- Joining Process
- Exit Policy

Module 3 — Research Agent

This agent performs external research using internet search and Wikipedia, collecting and summarizing information from multiple online sources.

"Research Microsoft's latest AI products."

"Latest Generative AI trends"

Module 4 — Email Agent

The Email Agent should draft professional emails, read the inbox, summarize recent emails, and send emails through Gmail.

"Draft an email requesting three days leave."

"Summarize my latest five emails."

Module 5 — Document Agent

This agent should search uploaded documents, answer questions from PDFs, retrieve company manuals, and explain company policies. The Document Agent must use ChromaDB.

Module 6 — Coordinator Agent

The Coordinator Agent is responsible for routing employee requests to the correct agent.

Module 7 — Sequential Workflow

Students should build a sequential pipeline: User Request → Research → Summarize → Generate Email → Send Email → Store Conversation.

Module 8 — Parallel Processing

The application should execute multiple independent tasks simultaneously, with each research task completing independently before the final report is combined.

“Research Google, Microsoft, Amazon, and OpenAI.”

Module 9 — Conditional Routing

The assistant should automatically determine which agent should process the request, implemented using RunnableBranch.

Module 10 — Company Knowledge Base

Users should upload HR Policies, the Employee Handbook, Company Guidelines, Project Documentation, and SOP Documents. Students should implement a PDF Loader, Text Splitter, ChromaDB, and Retriever.

Module 11 — Memory

Short-term memory maintains the current conversation. Persistent memory stores conversations using SQLite. Long-term memory retains employee name, department, frequently asked questions, preferred email style, and frequently accessed documents.

Module 12 — Structured Outputs

All generated reports should follow a structured format — Task Summary, Department, Priority, Actions Taken, Pending Actions, Recommended Next Steps — validated using Pydantic models.

Module 13 — Python Tool

Create a Python execution tool capable of performing calculations, formatting tables, creating reports, and processing CSV files.

“Calculate employee attendance percentage.”

“Create a summary table.”

Module 14 — Google Drive Integration (Optional)

Allow users to upload reports, search previous reports, and store generated documents.

Routing Examples

Leave policy



HR Agent

Current AI News



Research Agent

Send Mail



Email Agent

Company Handbook



Document Agent

Streamlit Application Requirements

Sidebar

- Upload Company Documents
- Build Knowledge Base
- View Uploaded Documents
- Previous Conversations
- Clear Chat

Main Screen

- Professional Chat Interface
- Conversation History
- Agent Responses

Expandable Sections

- Selected Agent
- Tool Execution Summary
- Retrieved Documents
- Generated Report

Download Options

- TXT
- PDF

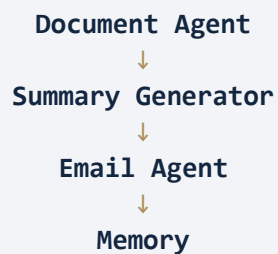
Recommended LangChain Components

▶ Chat Models	▶ Prompt Templates
▶ ChatPromptTemplate	▶ Pydantic Structured Output
▶ RunnableLambda	▶ RunnableParallel
▶ RunnableBranch	▶ create_agent()
▶ Custom Tools	▶ Gmail Toolkit
▶ Python REPL Tool	▶ ChromaDB Retriever
▶ SQLite Memory	

Example Workflows

Scenario 1

“Summarize the employee handbook and email it to HR.”



Scenario 2

“Research Google's AI products and compare them with Microsoft.”



Scenario 3

“Remember that I belong to the Finance Department.”



Scenario 4

“What department do I belong to?”



Expected Deliverables

▶ Source Code	▶ Streamlit Application
▶ GitHub Repository	▶ Project Documentation
▶ Requirements File	▶ Sample Company Documents
▶ README	▶ Demo Video (Optional)

CAPSTONE PROJECT 3

AI Investment & Financial Research Assistant

Case Study

AlphaVest Capital is a financial consulting and investment advisory firm that assists clients in making informed investment decisions. Analysts spend hours every day researching company fundamentals, market news, quarterly reports, economic indicators, and industry trends before preparing investment reports.

The company wants to develop an AI Investment & Financial Research Assistant that automates the research process — gathering information from multiple sources, analyzing financial reports, comparing companies, generating investment insights, remembering client preferences, and preparing professional research reports.

The solution should be developed using LangChain and deployed using Streamlit.

Business Problem

A financial analyst typically performs the following tasks:

- Search for the latest financial news
- Read annual and quarterly reports
- Compare multiple companies
- Summarize earnings reports
- Analyze industry trends
- Prepare investment recommendations
- Email reports to clients

The objective is to automate this workflow using AI agents.

Project Objective

Develop an AI-powered Financial Research Assistant capable of:

- Performing financial research
- Searching the internet
- Reading uploaded annual reports
- Comparing multiple companies
- Remembering investor preferences
- Preparing structured investment reports
- Sending reports through Gmail
- Running as a Streamlit application

System Architecture



Functional Requirements

Module 1 — AI Financial Assistant

Provide a conversational interface where investors can ask questions naturally.

"Research NVIDIA."

"Should I compare Microsoft and Google?"

"Summarize Tesla's annual report."

Module 2 — Financial News Agent

The assistant should search for current financial news using internet search tools.

"Google Stock"

"NVIDIA Earnings"

"Microsoft Quarterly Results"

"AI Industry News"

"Semiconductor Market"

Module 3 — Company Research Agent

The assistant should perform company-level research, combining internet search and Wikipedia.

- Business Overview
- Products
- Competitors
- Revenue Sources
- Recent Announcements

Module 4 — Annual Report Analysis (RAG)

Users should upload Annual Reports, Quarterly Reports, Investor Presentations, and Financial Statements. The assistant should answer questions using the uploaded documents.

“Summarize the risk factors.”

“Explain revenue growth.”

“What are the future plans?”

Module 5 — Multi-Company Comparison

Users should be able to compare multiple companies, with research performed in parallel before the comparison is prepared. Students should implement `RunnableParallel`.

“Compare Google, Microsoft, Amazon, and Meta.”

Module 6 — Investment Report Generator

Generate reports using structured outputs — Company Overview, Industry, Business Model, Latest News, Strengths, Weaknesses, Financial Highlights, Growth Opportunities, Potential Risks, and Investment Summary — using `Pydantic`.

Module 7 — Sequential Workflow

Pipeline: Research → Read PDF → Merge Information → Analyze → Generate Report → Generate Executive Summary → Email Report.

Module 8 — Conditional Routing

Students should implement `RunnableBranch` to route requests to the correct agent.

Module 9 — Memory

Short-term memory tracks the ongoing discussion. Persistent memory stores conversation history using `SQLite`. Long-term memory retains client name, investment interests, preferred industries, risk profile, and frequently researched companies.

Module 10 — Python Tool

Students should build a Python tool capable of financial calculations, growth percentage calculations, investment comparisons, ROI calculations, CAGR calculations, and table generation.

“Calculate CAGR.”

“Calculate annual growth.”

“Create a comparison table.”

Module 11 — Gmail Integration

Generate or send investment reports to the client.

“Email today's investment report to the client.”

Module 12 — Google Drive Integration (Optional)

Allow users to upload annual reports, store generated reports, and retrieve historical reports.

Routing Examples

Latest Google Stock News



News Agent

Analyze Tesla Annual Report



PDF Agent

Compare Microsoft and Amazon



Parallel Research Agent

Streamlit Application Requirements

Sidebar

- Upload Annual Reports
- Upload Quarterly Reports
- Build Knowledge Base
- View Uploaded Reports
- Previous Conversations
- Clear Chat

Main Screen

- Professional Chat Interface
- Generated Investment Reports
- Comparison Tables

Expandable Sections

- Latest News
- Retrieved PDF Chunks
- Financial Calculations
- Final Recommendation

Download Options

- TXT

- PDF

Recommended LangChain Components

▶ Chat Models	▶ Prompt Templates
▶ ChatPromptTemplate	▶ Pydantic Structured Output
▶ RunnableLambda	▶ RunnableParallel
▶ RunnableBranch	▶ create_agent()
▶ Custom Tools	▶ DuckDuckGo Search
▶ Wikipedia Tool	▶ Python REPL Tool
▶ ChromaDB Retriever	▶ SQLite Memory

Example Workflows

Scenario 1

“Research NVIDIA and summarize the latest AI announcements.”



Scenario 2

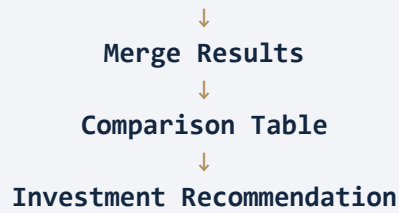
“Analyze Google's annual report and compare it with the latest news.”



Scenario 3

“Compare Microsoft, Google, Amazon, and Meta.”





Scenario 4

"Remember that I prefer low-risk technology investments."



Scenario 5

"Generate today's investment report and email it to my client."



Suggested Tools

▶ Internet Search Tool	▶ Wikipedia Tool
▶ Python Calculation Tool	▶ Gmail Toolkit
▶ Google Drive Toolkit (Optional)	▶ Weather Tool (Optional)
▶ Custom Financial Calculator Tool	▶ RAG PDF Tool

Expected Deliverables

▶ Streamlit Application	▶ Source Code
▶ GitHub Repository	▶ README
▶ Sample Annual Reports	▶ Project Documentation
▶ Requirements File	

CAPSTONE PROJECT 4

AI Personal Knowledge & Productivity Assistant

"Second Brain"

Case Study

Productive Minds Consulting is a knowledge-driven organization where consultants, researchers, trainers, and project managers work with thousands of documents, emails, meeting notes, presentations, and research papers every month.

Employees often struggle to find information from old documents, remember previous conversations, organize notes, search meeting minutes, summarize emails, create reports from multiple documents, manage personal tasks, and keep track of project deadlines.

The company wants to build a Personal AI Knowledge & Productivity Assistant that acts as a digital "Second Brain," understanding user requests, remembering personal preferences, searching across uploaded documents, organizing knowledge, summarizing information, managing tasks, and assisting with day-to-day productivity.

The application must be developed using LangChain and deployed using Streamlit.

Project Objective

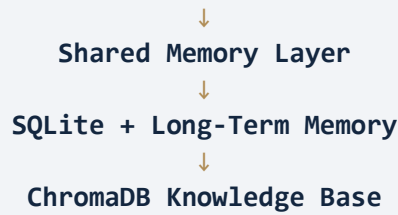
Develop an AI-powered productivity assistant capable of becoming a user's centralized knowledge management system, intelligently combining memory, document retrieval, internet search, email support, task management, and knowledge organization into one application.

Business Scenario

A consultant starts their day by asking: "Summarize yesterday's emails, remind me about my pending tasks, search my uploaded project documents for Client ABC, research today's AI news, and prepare a daily briefing." Instead of using multiple applications, one AI assistant completes the entire workflow.

System Architecture





Functional Requirements

Module 1 — AI Productivity Chat

Provide a conversational assistant capable of handling productivity-related requests.

“Summarize today's work.”

“Find my Machine Learning notes.”

“Prepare my daily briefing.”

Module 2 — Knowledge Base Agent (RAG)

Users should upload PDFs, TXT files, study notes, meeting minutes, project documents, and user manuals. The assistant should answer questions using the uploaded knowledge.

“Explain the architecture discussed in Project Alpha.”

“Summarize Meeting Minutes from Monday.”

Module 3 — Research Agent

The assistant should perform internet research whenever external information is required, integrating internet search and Wikipedia.

- AI News
- Technology Trends
- Competitor Information
- Industry Updates

Module 4 — Email Agent

The assistant should summarize the inbox, draft emails, send emails, reply to emails, and generate professional email content.

“Summarize my latest five emails.”

“Draft a follow-up email for yesterday's client meeting.”

Module 5 — Notes Agent

Students should build custom tools to manage personal notes — create, read, search, delete, and update.

“Save a note about tomorrow's client presentation.”

“Show all AI-related notes.”

Module 6 — Task Management Agent

Students should implement a custom task manager — add, delete, mark complete, list pending tasks, and search tasks.

“Add a task to prepare tomorrow's workshop.”

“Show today's pending tasks.”

Module 7 — Daily Briefing Generator

The assistant should generate a daily productivity summary covering pending tasks, recent emails, today's AI news, recent notes, and suggested priorities — combining multiple agents into a single workflow.

“Prepare today's briefing.”

Module 8 — Sequential Workflow

Pipeline: User Request → Collect Notes → Search Documents → Research Internet → Summarize Emails → Generate Final Briefing → Store Conversation.

Module 9 — Parallel Processing

Students should execute multiple independent tasks simultaneously, implemented using `RunnableParallel`.

“Generate today's briefing.”

Module 10 — Conditional Routing

The assistant should automatically determine which module should process each request, using `RunnableBranch`.

Module 11 — Memory

Short-term memory tracks the current conversation. Persistent memory stores conversations using SQLite. Long-term memory retains user name, preferred report style, frequently used topics, favorite technologies, frequently accessed documents, and frequently used email templates.

Module 12 — Structured Output

Daily Briefings should follow a fixed structure — Daily Summary, Important Emails, Pending Tasks, Knowledge Base Highlights, Latest Research, Recommendations, Next Actions — validated using Pydantic.

Module 13 — Python Tool

Students should build a Python execution tool capable of creating summary tables, CSV analysis, generating charts, simple calculations, and data processing.

“Analyze my task completion statistics.”

“Generate a chart of completed vs pending tasks.”

Module 14 — Google Drive Integration (Optional)

Allow users to upload project documents, search stored documents, archive reports, and back up notes.

Routing Examples

Find my project notes.



Notes Agent

Latest AI News



Research Agent

Summarize my emails.



Email Agent

Search uploaded documents.



Knowledge Agent

Streamlit Application Requirements

Sidebar

- Upload Documents
- Build Knowledge Base
- Manage Notes
- View Tasks
- Previous Conversations
- Clear Chat

Main Screen

- Professional Chat Interface
- Daily Briefings
- Research Results
- Knowledge Search

Expandable Sections

- Tool Execution Summary
- Retrieved Documents

- Email Summary
- Notes Retrieved
- Tasks Retrieved

Download Options

- Daily Briefing
- Notes
- Reports
- TXT
- PDF

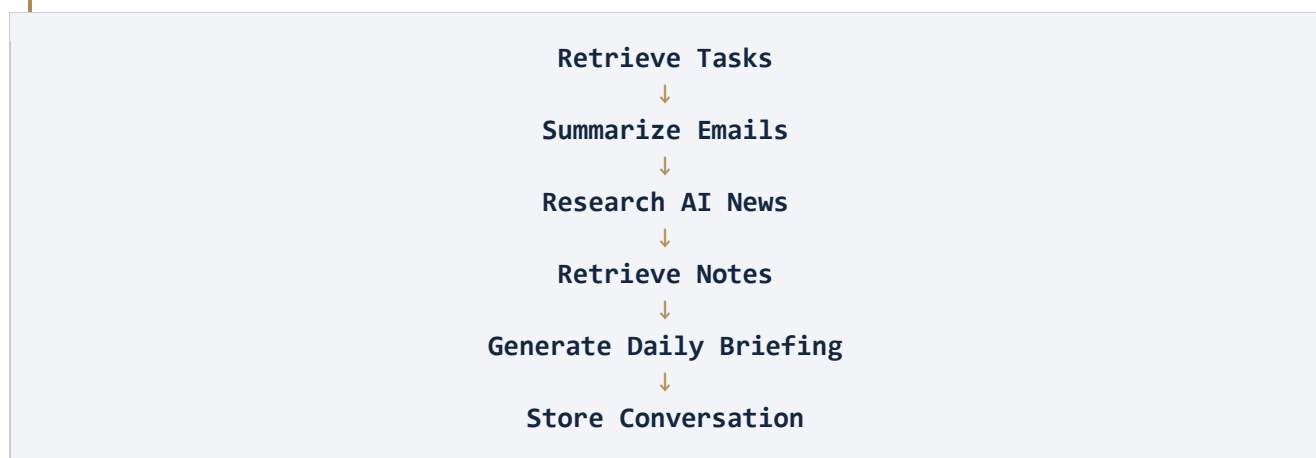
Recommended LangChain Components

▶ Chat Models	▶ Prompt Templates
▶ ChatPromptTemplate	▶ Structured Output
▶ RunnableLambda	▶ RunnableParallel
▶ RunnableBranch	▶ create_agent()
▶ Custom Tools	▶ Gmail Toolkit
▶ DuckDuckGo Search	▶ Wikipedia Tool
▶ Python REPL Tool	▶ ChromaDB Retriever
▶ SQLite Memory	

Example Workflows

Scenario 1

"Prepare my morning briefing."



Scenario 2

"Find my LangChain project notes and email them to my mentor."

Knowledge Agent



Notes Agent



Email Agent



Memory

Scenario 3

"Research today's AI news and compare it with my uploaded research papers."

Research Agent



Knowledge Agent



Merge Information



Generate Comparison Report

Scenario 4

"Remember that I prefer concise daily reports and I'm currently working on the Agentic AI course."

Long-Term Memory



Store User Preferences



Future Personalization

Scenario 5

"Analyze my weekly task completion and generate a productivity report."

Task Agent



Python Tool



Generate Statistics



Structured Report

Suggested Tools

Built-in Tools

- Internet Search Tool
- Wikipedia Tool
- Python REPL Tool
- Gmail Toolkit

Custom Tools

- Notes Manager
- Task Manager
- PDF Search Tool
- Daily Briefing Generator
- Productivity Statistics Tool

Optional Tools

- Google Drive Toolkit
- Weather Tool
- Calendar Reminder Tool (basic custom implementation)

Expected Deliverables

▶ Streamlit Application	▶ Complete Source Code
▶ GitHub Repository	▶ README Documentation
▶ Sample Knowledge Base Documents	▶ Notes Database
▶ Task Database	▶ Requirements File
▶ User Guide	